



Automatización

Clase N° 5: Introducción a HTML y CSS - Estructura de Páginas Web

¡Les damos la bienvenida!

 Vamos a comenzar a grabar la clase.

Índice

Clase 4

Introducción a Pytest y Automatización Básica

- Instalación de Pytest y preparación del proyecto
- Creación de casos de prueba
- Decoradores y aserciones
- Fixtures: qué son, para qué sirven y variantes
- Parametrización y markers personalizados
- Reporte HTML nativo
- Ejercicios prácticos.

Clase 5

Introducción a HTML y CSS – Estructura de Páginas Web

- Anatomía de un documento HTML
- Elementos HTML comunes (div, form, input, button, etc.)
- Atributos esenciales para automatización (id, class, name, value)
- Introducción a CSS
- DevTools
- Buenas prácticas de selectores

Clase 6

DOM para Automatización

- ¿Qué es el DOM y por qué nos importa en QA?
- Recorrido del DOM con selectores CSS
- XPath: otra vía para llegar al elemento correcto
- Estrategias de localización: del id al XPath relativo
- Ejercicios

¿Qué vamos a ver hoy?

1

Estructura HTML

Anatomía de una página web y sus componentes fundamentales.

2

Elementos y atributos

Identificadores clave para automatización: id, class y name.

3

Estilos CSS

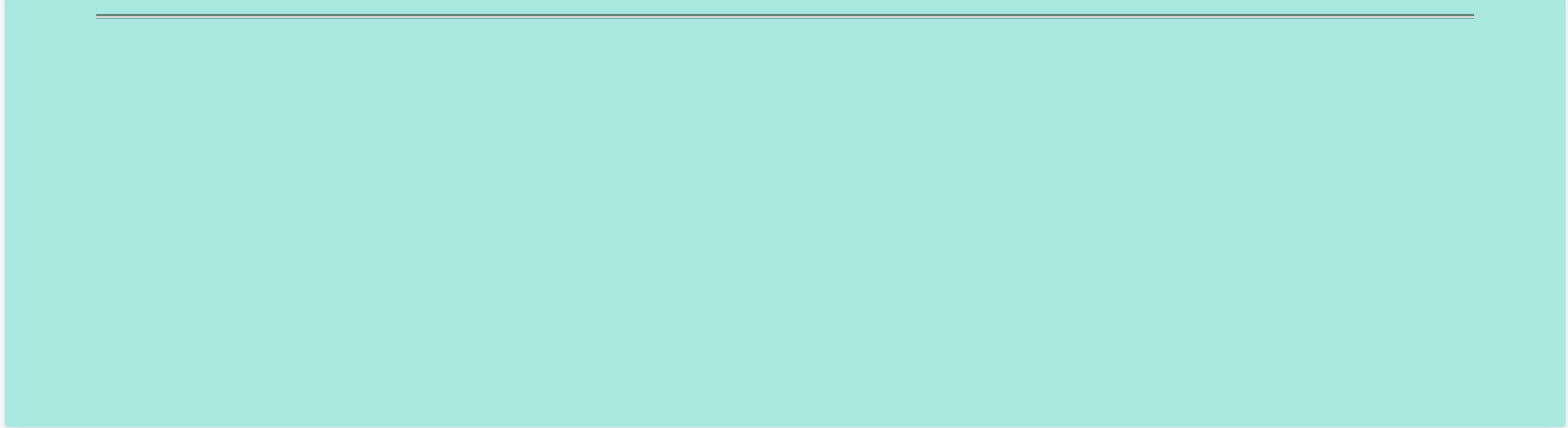
Tres formas de aplicar estilos: en línea, incrustado y externo.

4

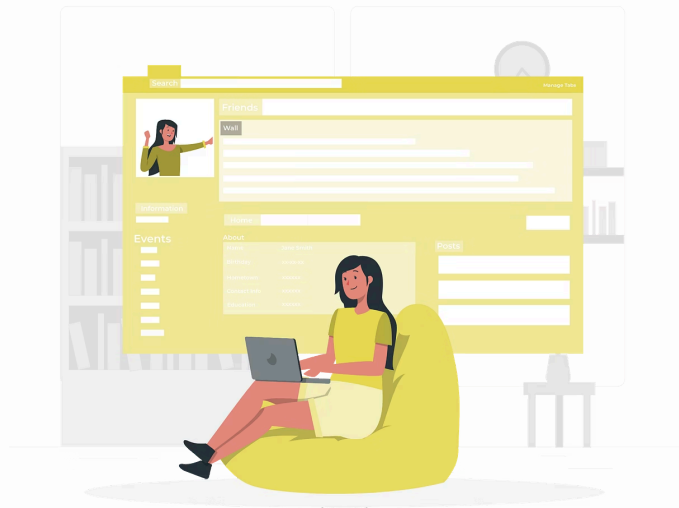
Selectores y DevTools

Jerarquía de selectores y herramientas para inspeccionar el DOM.

HTML



Anatomía de un documento HTML



Antes de poder automatizar una interfaz, necesitamos comprender cómo está construida.

HTML define la estructura de la página —es como el esqueleto del sitio—, mientras que **CSS** se encarga del diseño visual: colores, tamaños, tipografías y disposición.

Estructura Básica

Un archivo HTML funciona como el esqueleto de la página. Comienza con la declaración que indica al navegador que usará HTML5.

HTML

HTML (HyperText Markup Language) es el lenguaje que define la estructura de una página web.

Cada página tiene un “esqueleto” hecho con etiquetas que indican qué es cada parte: un título, un párrafo, un formulario, etc.



Estructura Base

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <title>Página de ejemplo</title>
  </head>
  <body>
    <h1>¡Hola TalentoLab!</h1>
    <p>Esta es nuestra primera página.</p>
  </body>
</html>
```

- **<!DOCTYPE html>**: indica al navegador que usará HTML5.
- **<head>**: contiene metadatos, hojas de estilo y scripts.
- **<body>**: todo lo que el usuario ve e interactúa.

Elementos HTML Comunes

Etiqueta	Uso principal	Mini-ejemplo
header, nav, main, section, article, footer	Semantización de la estructura principal de la web	

div	Contenedor genérico	div class="tarjeta"
form	Agrupar campos de entrada	form action="/alta"
input	Campo de texto, checkbox, etc.	input type="email" name="correo"
button	Botón de acción	button>Enviar
a	Enlace a otra URL	a href="/inicio">Inicio
ul / li	Listas	 • Item

⊗ Aunque existen etiquetas semánticas como header, nav y section, muchas aplicaciones aún utilizan div como ladrillo base. Por eso verás muchos selectores apuntando a ellos en tus pruebas de automatización.

Para más conocer mas información, etiquetas y sus usos ingresá a:

👉 [MDN Developer Mozilla](#)

Atributos Esenciales para Automatización



id

Identificador único en la página. Perfecto para selectores robustos.

Ejemplo: **input id="email"**



class

Agrupar elementos que comparten estilo o función. Un elemento

puede tener varias clases. Ejemplo: **div class="alert error"**



name

Nombre que recibe el backend al enviar el formulario; Selenium también lo usa. Ejemplo: `input name="password"`



value

Valor inicial o etiqueta de un control. Ejemplo: `button value="guardar">Guardar`

 Consejo: elige id siempre que sea estable. Evitarás selectores frágiles que se rompan con cambios de diseño.

Definiendo estilos con CSS

Introducción a CSS

CSS se encarga de definir cómo se ve una página web, incluyendo aspectos como colores, fuentes, diseño y disposición de los elementos. Permite separar la estructura (HTML) del estilo (CSS), lo que facilita el diseño y mantenimiento de las páginas web.

Sintaxis:

```
selector{  
  propiedad:valor;  
}
```

Existen muchas formas de implementar CSS en nuestros proyectos, algunas mejores que otras dependiendo de las necesidades del mismo.

CSS



Estilos en línea

Aplicación de estilos directo a las etiquetas, ideal para pruebas rápidas pero no permanentes:

```
<h1 style="color: steelblue; font-size: 24px;">
  Título en línea</h1>
```

✓ Ventajas

- Se ve el resultado de inmediato
- Ideal para pruebas rápidas o ajustes puntuales
- Separa el código CSS del HTML
- Permite reutilizar estilos en múltiples elementos

✗ Desventajas

- Mezcla estructura y presentación
- Dificulta el mantenimiento y la reutilización de estilos
- No aprovecha el caché del navegador
- Complica proyectos grandes y multidisciplinarios

Bloque <style>

Bloque <style> dentro del head del proyecto.

```
<head>
  <style>
    h1 {
      color: steelblue;
      font-size: 24px;
    }
    .boton {
      background-color: #0275d8;
      color: white;
      padding: 10px;
      border: none;
      border-radius: 4px;
    }
  </style>
</head>
```

✓ Aplicación

Ideal para prototipos rápidos o demos pequeños, definiendo estilos en un solo lugar.

✗ Desventajas

- No recomendado para proyectos grandes porque mezcla estructura con estilo.
- No es escalable.

CSS en Archivo Externo

¿Por qué usarlo?

Es la mejor práctica para aplicar estilos en desarrollo web.

Separa el contenido (HTML) de la presentación (CSS).

- Reutilizable en múltiples páginas

Paso 1: Crear archivo CSS

```
/* estilos.css */
h1 {
  color: darkcyan;
  font-family: 'Segoe UI', sans-serif;
}

.boton {
```

Paso 2: Vincular desde HTML

```
<head>
  <link rel="stylesheet"
  href="estilos.css">
</head>
```

- Mejora el rendimiento por caché del navegador
- Facilita la escalabilidad y mantenimiento



✓ Recomendado para la mayoría de los proyectos.

Separar estructura (HTML) de estilo (CSS) mejora el mantenimiento, trabajo en equipo y rendimiento.

```
background-color: #0275d8;  
color: white;  
padding: 8px 16px;  
border: none;  
border-radius: 4px;  
}
```

Esta vinculación permite al navegador cargar los estilos separadamente.

Selectores básicos

Ejemplos de selectores Básicos en CSS

Selector de Elemento

Selecciona todos los elementos del tipo especificado.

```
p {  
  color: blue;  
}
```

Este selector afecta a todos los párrafos del documento.

Selector de Clase

Selecciona elementos con una clase específica.

```
.destacado {  
  font-weight: bold;  
}
```

Se aplica a cualquier elemento con `class="destacado"`.

Selector de ID

Selecciona un elemento único con el ID especificado.

```
#cabecera {  
  background: gray;  
}
```

Alta especificidad, debe ser único en todo el documento.

Especificidad y Jerarquía de Selectores

1

Estilos en Línea

Máxima especificidad (style="color: red")

2

Selector de ID

Mayor especificidad (#principal, #usuario)

3

Selector de Clase

Especificidad media (.boton, .tarjeta)

4

Selector de Etiqueta

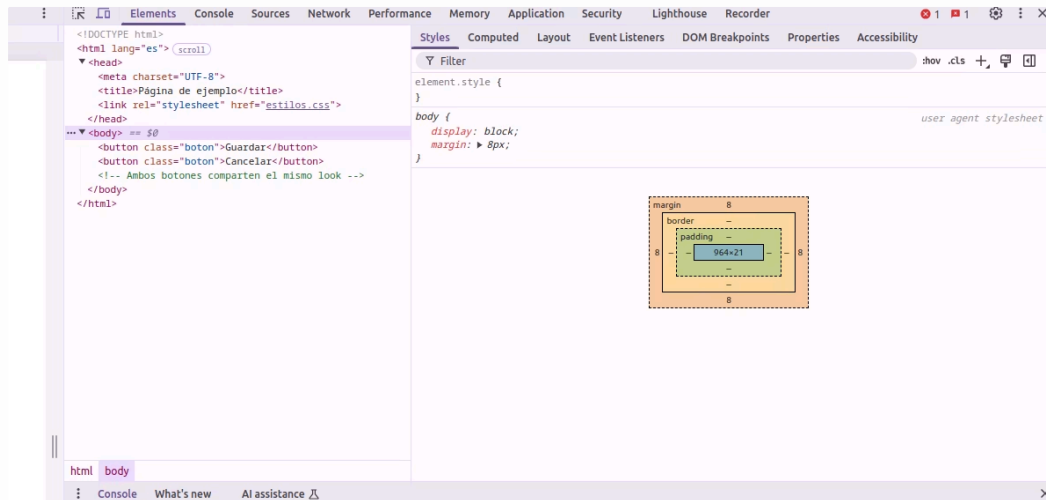
Menor especificidad (p, h1, div)

 Si un mismo elemento coincide con varios selectores, el navegador elige el más "específico". Es decir, un estilo definido con #id sobrescribirá al de .clase, y ambos sobrescribirán al de una etiqueta. Conocer esta jerarquía te evitará sorpresas cuando un cambio parezca "no hacer efecto".

DevTools

**Herramientas de desarrollo del navegador
(DevTools)**

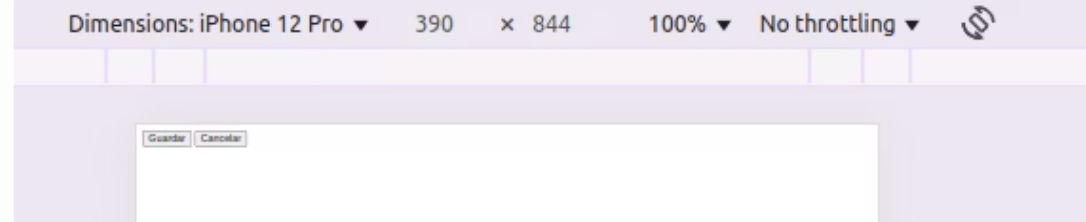
¡Tu microscopio para ver el DOM en vivo! Ábrelo con **F12** o clic derecho → *Inspeccionar*.



Pestañas principales

- **Elements:** árbol del DOM y estilos aplicados.
- **Styles:** reglas CSS; desmarca para ver cambios inmediatos.
- **Console:** ejecuta JS y revisa errores.
- **Network:** solicitudes HTTP (excelente para API debugging).
- **Performance y Lighthouse:** métricas de carga y accesibilidad.

Modo Dispositivo en DevTools



Activación

Haz clic en el icono de dispositivo móvil en la barra de herramientas de DevTools para activar el modo de emulación de dispositivos.

Selección de Dispositivo

Elige entre presets predefinidos como iPhone 12 Pro o Pixel 6, o define tu propio tamaño personalizado para probar diferentes resoluciones.

Orientación y Zoom

Cambia entre orientación vertical y horizontal, y ajusta el DPR (Device Pixel Ratio) para simular diferentes densidades de pantalla.

i El modo dispositivo es fundamental para validar que los selectores y estilos sean consistentes en vistas móviles. Permite detectar problemas de diseño responsivo y asegurar que la automatización funcione correctamente en diferentes tamaños de pantalla.

Inspección y Copia de Selectores (DevTools)

1 Seleccionar un elemento

Haz clic derecho sobre cualquier elemento de la web

2 Copiar el selector CSS

En el panel Elements, haz clic derecho sobre el elemento.

3 Probar el selector en la consola

En la pestaña Console, escribe:

y selecciona "Inspeccionar".

El panel Elements mostrará el HTML con el elemento resaltado.

Selecciona "Copy → Copy selector" para obtener un selector único como "body > div > h1".

`document.querySelector('tu-selector')`.

Si es válido, verás el elemento resaltado. Si devuelve null, el selector es incorrecto.

Buenas Prácticas de Selectores

Prioriza Atributos Únicos

Usa id cuando exista, ya que proporciona la forma más directa y estable de identificar un elemento. Los selectores basados en id son menos propensos a romperse con cambios de diseño.

Clases Semánticas

Prefiere clases con nombres descriptivos (`.form-login`, `.btn-primary`) que indiquen la función del elemento, no solo su apariencia. Esto hace que los selectores sean más comprensibles.

Evita Complejidad Innecesaria

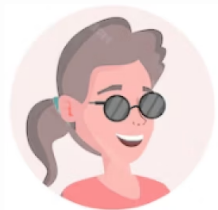
Evita rutas largas y selectores basados en posición (`nth-child`) a menos que no haya alternativa. Estos selectores son frágiles y pueden romperse fácilmente con cambios en la estructura.

- ① Siguiendo estas prácticas, crearás selectores robustos que resistirán cambios en el diseño y facilitarán la automatización de pruebas con herramientas como Selenium.

Ejercicios Prácticos

¡Otro día en Talento Lab!

El equipo de Talento Lab va a mostrar la nueva calculadora web en la demo interna de la semana próxima. Silvia necesita que el formulario esté listo para ser automatizado con Selenium y te pide:



“Quiero un HTML limpio, sin JavaScript, con todos los atributos id y name únicos. Además, dejame un documento con los selectores que usarás en tus scripts.”



Matías comenta:

“Olvidate de la lógica por ahora; ya la validamos con Pytest. Solo maquetá el front y anotá buenos selectores. Después integraremos Selenium.”

Ejercicio Práctico

Obligatorio

Objetivos:

- Construir la vista estática de la calculadora (HTML + CSS externo).
- **Etiquetar cada campo con identificadores estables (id, name, clases semánticas).**
- Practicar DevTools: inspeccionar elementos, copiar selectores y comprobar que apunten a unívocos.
- Crear selectors.md — tu futura hoja de ruta para los tests UI.

Entregables y estructura de carpetas

Clase05/

| index.html ← estructura HTML 100 % estática

| estilos.css ← hoja de estilos externa

| selectors.md ← tabla de selectores (Markdown)

└─ README.md ← breve guía de ejecución y verificación

Ejercicio Práctico

Obligatorio

1) Crea un archivo index.html con la estructura mínima de una página web

2) Hoja de estilos:

- Agregá un archivo estilos.css con estilos básicos (reset, colores, clases utilitarias, etc.). Podés usar una plantilla base o una que hayas trabajado previamente.

3) Abrir con DevTools:

- Desde el navegador, presioná Ctrl + O y abrí el archivo index.html.
- Luego, usá F12 o clic derecho → *Inspeccionar* para acceder a las DevTools.

4) Verificación de controles:

- Inspeccioná cada input, botón o campo del formulario.
- Asegurate de que cada uno tenga un atributo id **único**.
- Si un elemento no tiene id, agregalo y guardá los cambios.

5) Test rápido de selectores CSS:

- Copiá el selector CSS de cada elemento y probalo en la consola del navegador con este comando:
`document.querySelector('#num1').style.border = '2px solid red';`



● Si el borde se pinta de rojo, el selector es correcto.



Ejercicio Práctico

Obligatorio

6) Documentación de selectores (selectors.md):

Elemento	Atributo usado	Selector CSS
Campo Número 1	id	#num1
Campo Email	name	input[name="email"]
Botón Enviar	class	.btn.enviar
Checkbox Términos	id	#acepta-condiciones
Error debajo de email	class	.error-msg.email

